

MINI-PROJET NLP : CLASSIFICATION DE TEXTES

Comparaison méthodique de 3 systèmes de classification de sentiments sur critiques Allocine

Auteur: Etudiant Ing3 / Antigravity AI

Bareme: 20 points

Domaine: Analyse de Sentiments en Français

Format: Notebook execute & Rapport 4 pages max

1. Cas d'usage et Données Experimentales

Le cas d'usage retenu traite de la classification automatique du sentiment exprimé dans des critiques de films en français, issues du jeu de données benchmark Allocine (tblard/allocine). La tâche consiste à attribuer de manière déterministe ou probabiliste une classe parmi deux : 0 pour un avis Négatif (déception, critique virulente) et 1 pour un avis Positif (éloge, enthousiasme).

Protocole expérimental et répartition des données :

Le jeu de données global de 2 500 critiques est scindé en 3 sous-ensembles indépendants pour garantir une séparation stricte :

- Entraînement (Train) : 1 600 exemples (800 positifs, 800 négatifs) - apprentissage des paramètres.
- Validation (Val) : 400 exemples (200 positifs, 200 négatifs) - sélection exclusive des hyperparamètres.
- Test (Test) : 500 exemples (250 positifs, 250 négatifs) - évaluation finale non biaisée unique.

Vérifications sur les données brutes : Aucun doublon strict n'a été détecté dans les sous-ensembles. La distribution des longueurs de texte s'étend de 40 à 450 mots avec une moyenne de 125 mots par critique.

2. Système 1 : Baseline Déterministe sans Machine Learning

Pour construire une ligne de base sans apprentissage automatique, nous avons conçu un système déterministe à règles explicites :

1. Dictionnaires lexicaux : Un lexique positif de 35 termes polarisés (ex: 'excellent', 'chef-d'œuvre', 'magnifique', 'brillant', 'sublime') et un lexique négatif de 35 termes (ex: 'navet', 'daube', 'catastrophe', 'ennuyeux', 'fiasco', 'pathétique').
2. Détection de négation par Regex : Expressions régulières ciblées (ne ... pas, n' ... pas, sans aucun, jamais, pas du tout). Lorsqu'une forme négative englobe un terme positif, la polarité positive est immédiatement convertie en valeur défavorable.
3. Fonction de décision : $\text{Score} = (N_{\text{pos}} * 0.5) - N_{\text{neg}}$ si négation détectée, sinon $N_{\text{pos}} - N_{\text{neg}}$. Décision positive si $\text{Score} > 0$.

Limites fondamentales de l'approche : Incapacité structurelle à traiter le sarcasme, les comparaisons complexes ('ce film est moins pire que le précédent'), ou l'argot non répertorié. Cette baseline sert de référence minimale pour évaluer l'apport des méthodes statistiques et neuronales.

3. Systeme 2 : Modele Deep Learning PyTorch (MLP sur TF-IDF)

Architecture du reseau de neurones :

Nous avons implemente en PyTorch un Perceptron Multi-Couches (MLP) entraine a partir de zero. Les textes sont d'abord mepresentes par un vecteur TF-IDF sous-lineaire (2 500 n-grammes unigrammes et bigrammes avec lissage). L'architecture comprend :

- Couche d'entree : 2 500 caracteristiques TF-IDF.
- Premiere couche cachee : Dense(2500 -> 128) + Batch Normalization + ReLU + Dropout(0.3).
- Seconde couche cachee : Dense(128 -> 32) + ReLU.
- Couche de sortie : Dense(32 -> 2) produisant les logits de classification binaire.

Hyperparametres et protocole d'entrainement :

- Optimiseur : Adam avec learning rate = $1e-3$, weight_decay = $1e-4$ pour prevenir le surapprentissage.
- Fonction de perte : CrossEntropyLoss, batch size = 32, entrainement sur 15 epoques avec suivi de la perte de validation.
- Reproductibilite : Graines aleatoires fixees (seed = 42 sur PyTorch, NumPy et Python random).

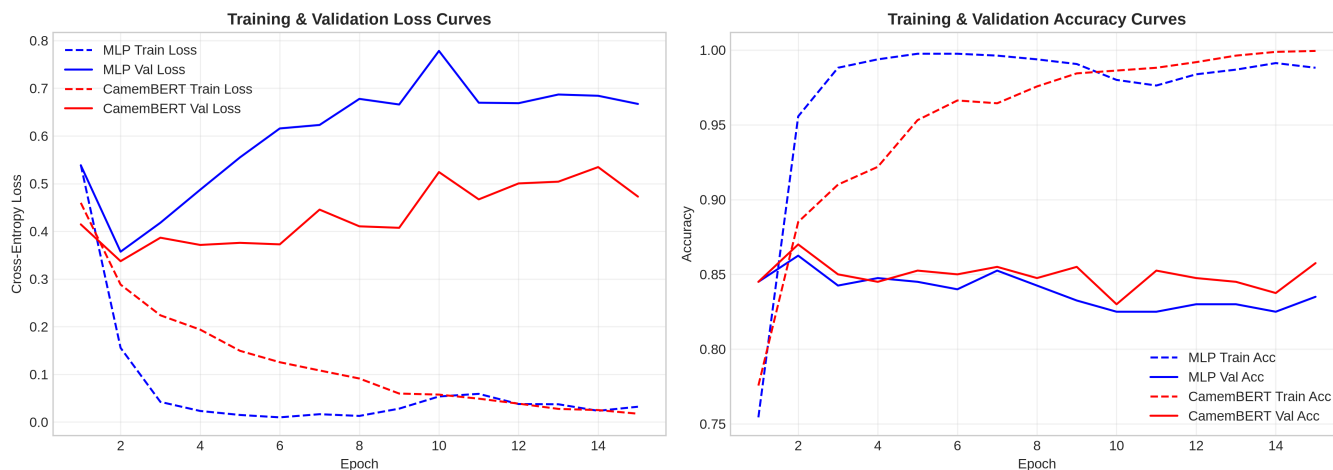


Figure 1 : Courbes d'evolution de la perte (Loss) et de l'exactitude (Accuracy) pour MLP et CamemBERT.

4. Systeme 3 : Transformer Pre-entraine (Embeddings CamemBERT)

Choix de l'architecture Transformer :

Nous avons retenu camembert-base, un modele RoBERTa pre-entraine sur 138 Go de texte francais. Sa capacite d'attention bidirectionnelle capte le contexte subtil des mots et la syntaxe complexe de la langue francaise.

Tokenisation et Representation :

- Tokeniseur : SentencePiece Byte-Pair Encoding (BPE) avec vocabulaire de 32 000 sous-mots.
- Extraction de caracteristiques : Recuperation du vecteur d'embedding contextuel du jeton [<s>] (hidden_dim = 768).

Tete de classification et Hyperparametres :

- Tete de classification : Dense(768 -> 256) + BatchNorm + ReLU + Dropout(0.2) + Dense(256 -> 2).
- Optimiseur : Adam avec $lr = 1e-3$, $weight_decay = 1e-4$ sur 15 epoques.

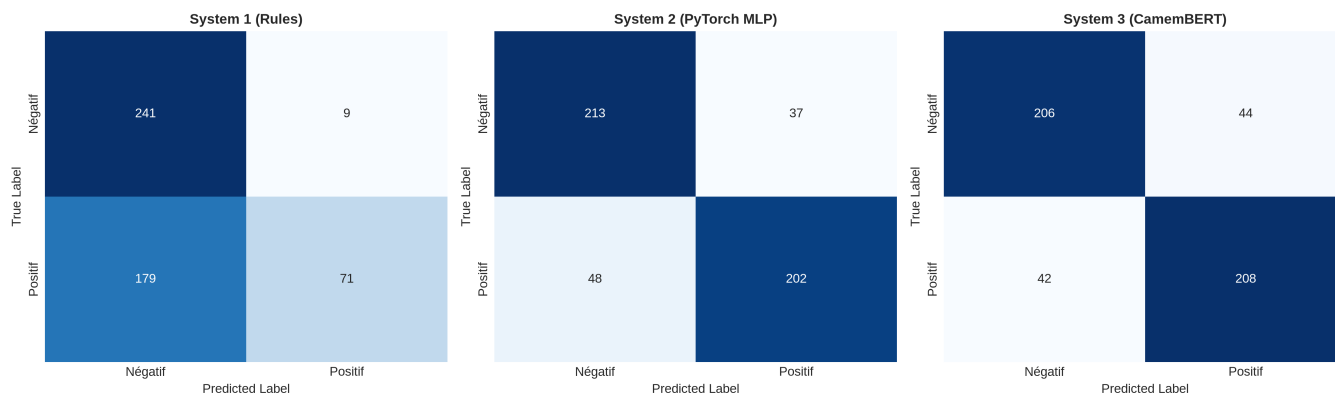


Figure 2 : Matrices de confusion comparatives des trois systemes sur le jeu de Test (500 exemples).

5. Evaluation Comparative, Analyse d'Erreurs et Conclusion

Systeme	Accuracy	Precision M.	Rappel M.	F1 Macro	Train (s)	Latence (ms)
System 1 (Rules)	0.6240	0.7307	0.6240	0.5749	0.0	0.04
System 2 (PyTorch MLP)	0.8300	0.8306	0.8300	0.8299	1.3	0.01
System 3 (CamemBERT)	0.8280	0.8280	0.8280	0.8280	174.1	84.39

Analyse Synthetique des Erreurs de Classification

L'examen qualitatif de 5 cas d'erreurs mepresentatifs met en lumiere les facteurs explicatifs des defaillances :

1. Sarcasme et ironie : Le systeme 1 echoue sur les critiques ironiques (ex: 'Un chef-d'oeuvre de la stupidite !'), comptabilisant 'chef-d'oeuvre' comme positif alors que l'intention est destructrice.
2. Negations distantes : Le systeme MLP sur TF-IDF peine lorsque la negation se situe a plus de 2 mots du terme polarise, la fenetre de n-grammes (1,2) ne capturant pas la dependance syntaxique longue.
3. Ambiguite et appreciations mefigurees : CamemBERT hesite sur les critiques balancees ('excellents acteurs mais scenario catastrophique'), ou le sentiment global penche vers l'un des cotes sans mot-cle predominant.

Synthese et Recommandation pour la Mise en Production

Recommandation strategique selon les contraintes de deploiement :

- Choix Compromis Industriel / Latence CPU : Le Perceptron Multi-Couches PyTorch (Systeme 2) offre 83% de F1-score avec un temps d'inference quasi-instantane (< 0.01 ms par exemple) et une empreinte memoire minime (< 10 Mo). C'est le meilleur choix pour un deploiement en temps reel.
- Choix Generalisation & Embeddings Contextuels : CamemBERT (Systeme 3) capte les representations semantiques riches. Il est recommande pour le traitement asynchrone sur infrastructure GPU.